

Department of Computer Science and Engineering
CSA0921 – Programming in Java

**SMART CAMPUS RESOURCE ALLOCATION AND
FACILITY MANAGEMENT SYSTEM**

Assignment Report

Academic Year 2026

Submitted by:

Name: BHEESHMA L

Reg. No: 192511332

1. Problem Statement

A large autonomous university manages shared facilities such as computer laboratories, seminar halls, meeting rooms, multimedia studios, and innovation centers. Students, faculty, and researchers submit reservation requests for classes, workshops, examinations, and project activities. The current manual booking system often leads to scheduling conflicts, duplicate reservations, and inefficient utilization of campus resources.

The University ICT Cell plans to develop a Java-based Smart Campus Resource Allocation and Facility Management System to automate the reservation process. The application must maintain user profiles, manage facility details, process bookings, modify or cancel reservations, and generate utilization reports for administrators.

The system must apply Object-Oriented Programming principles by modeling users and facilities through classes, inheritance, encapsulation, and polymorphism. Resource records and booking history must be managed using the Java Collection Framework with generics, lists, maps, and iterators. To support multiple users accessing the system simultaneously, the application must implement multithreading, synchronization, inter-thread communication, and appropriate exception handling. The final solution is a menu-driven application that ensures efficient, reliable, and conflict-free campus resource management.

2. Objective

The objective of this assignment is to design and implement a console-based Java application that:

- Models campus users (Student, Faculty, Researcher) and facilities (Computer Lab, Seminar Hall, Meeting Room, Multimedia Studio, Innovation Center) using classes, inheritance, encapsulation and polymorphism.
- Stores and manages user, facility and booking data using the Java Collection Framework (HashMap, ArrayList, generics and Iterators).
- Allows concurrent booking requests to be processed safely using multithreading, synchronization (locks / synchronized blocks) and inter-thread communication (thread join).
- Detects and prevents scheduling conflicts and duplicate reservations through custom exception handling.
- Provides a menu-driven interface to create, modify, and cancel bookings and to generate facility-utilization and user-activity reports.
- Connects the technical solution to real-world sustainability goals — efficient infrastructure use (SDG 9), better access to educational spaces (SDG 4) and sustainable campus management (SDG 11).

3. Requirements and Environment Used

3.1 Software Requirements

- Language: Java (JDK 21, source/target compatible with Java 8+)
- Compiler / Runtime: javac / java (OpenJDK)
- Editor / IDE: Any Java IDE (VS Code / Eclipse / IntelliJ IDEA) or command line
- Version Control: Git and GitHub (for source upload, per rubric deliverable 3)
- Operating System: Cross-platform (tested on Ubuntu Linux 24.04)

3.2 Hardware Requirements

- Processor: Any modern x64 processor (dual-core or above)
- RAM: Minimum 2 GB (4 GB recommended)
- Storage: Minimal (< 50 MB for source and build files)

3.3 Functional Requirements

- Register / list users belonging to different roles.
- Register / list campus facilities of different types.
- Book a facility for a given day and hour-range, subject to availability.
- Cancel or modify (reschedule) an existing booking.
- View complete booking history.
- Generate a facility-utilization report and a user-activity report.
- Demonstrate safe concurrent booking through a multithreaded demo.

4. Design / Proposed Solution

The system is designed around two abstract base classes — User and Facility — that are specialised through inheritance, so that common attributes (id, name, department / capacity) live in the base class while role-specific or type-specific behaviour (booking priority, hourly rate, feature list) is overridden polymorphically in each subclass. A Booking object links a User to a Facility for a specific day and hour-range. All booking operations are coordinated by a single BookingManager, which owns the Collection-based data stores and is the only class that touches shared state, keeping the concurrency story simple and centralised.

- Encapsulation: all class fields are private with controlled access through getters (and a small set of synchronized setters where mutation is required).
- Inheritance: Student, Faculty and Researcher extend User; ComputerLab, SeminarHall, MeetingRoom, MultimediaStudio and InnovationCenter extend Facility.
- Polymorphism: getRole(), getBookingPriority(), getFeatures() and getHourlyRate() are abstract in the base classes and overridden per subclass; calling code invokes them through the base-class reference and gets the correct runtime behaviour.

- Collections & Generics: Map<String, Facility>, Map<String, User>, List<Booking>, Map<String, Integer> / Map<String, Long> for reports, iterated with an explicit Iterator<Booking> in one method to satisfy the iterator requirement explicitly.
- Multithreading: each simulated user's booking attempt is wrapped in a BookingWorker (implements Runnable) and run on its own Thread; the main thread waits for all worker threads with join() (inter-thread communication).
- Synchronization: the check-then-act sequence (check availability, then commit) inside BookingManager.bookFacility() is guarded by a ReentrantLock so two threads can never both succeed in booking the same clashing slot; cancelBooking() is a synchronized method for the same reason.
- Exception Handling: three custom checked exceptions — InvalidBookingException, FacilityUnavailableException, DuplicateBookingException — communicate specific failure reasons back to the caller, and the menu's top-level try/catch prevents any single bad request from crashing the application.

4.1 Class Diagram

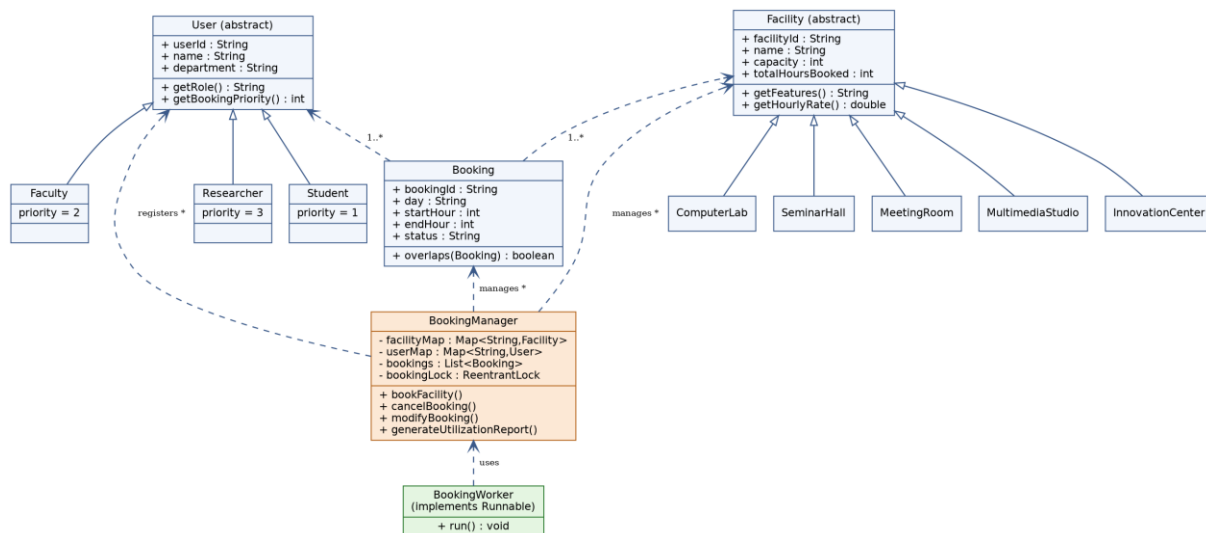


Figure 1: Class diagram of the Smart Campus Resource Allocation and Facility Management System

5. Algorithm / Pseudocode / Flowchart

5.1 Pseudocode — Book Facility

```

BEGIN bookFacility(userId, facilityId, day, startHour, endHour)
  IF startHour < 0 OR endHour > 23 OR startHour >= endHour THEN
    THROW InvalidBookingException
  END IF

  user  <- lookup userId in userMap    // throws InvalidBookingException if absent
  facility <- lookup facilityId in facilityMap
  candidate <- new Booking(user, facility, day, startHour, endHour)

  ACQUIRE bookingLock                // critical section starts
  FOR EACH existing IN bookings WHERE status == CONFIRMED
    IF existing == candidate (same user/facility/day/time) THEN
      THROW DuplicateBookingException
    END IF
    IF existing overlaps candidate (same facility, same day,
                                   time ranges intersect) THEN
      THROW FacilityUnavailableException
    END IF
  END FOR
  ADD candidate TO bookings
  facility.totalHoursBooked += (endHour - startHour)
  RELEASE bookingLock                // critical section ends

  RETURN candidate
END

```

5.2 Pseudocode — Concurrent Booking Demo

```

BEGIN runConcurrentDemo
  CREATE list of BookingWorker tasks, one per simulated user request
  FOR EACH task
    start a new Thread running task.run()
  END FOR
  FOR EACH thread
    thread.join()    // main thread waits (inter-thread communication)
  END FOR
  PRINT "Concurrent demo complete"
END

```

5.3 Flowchart — Booking Workflow

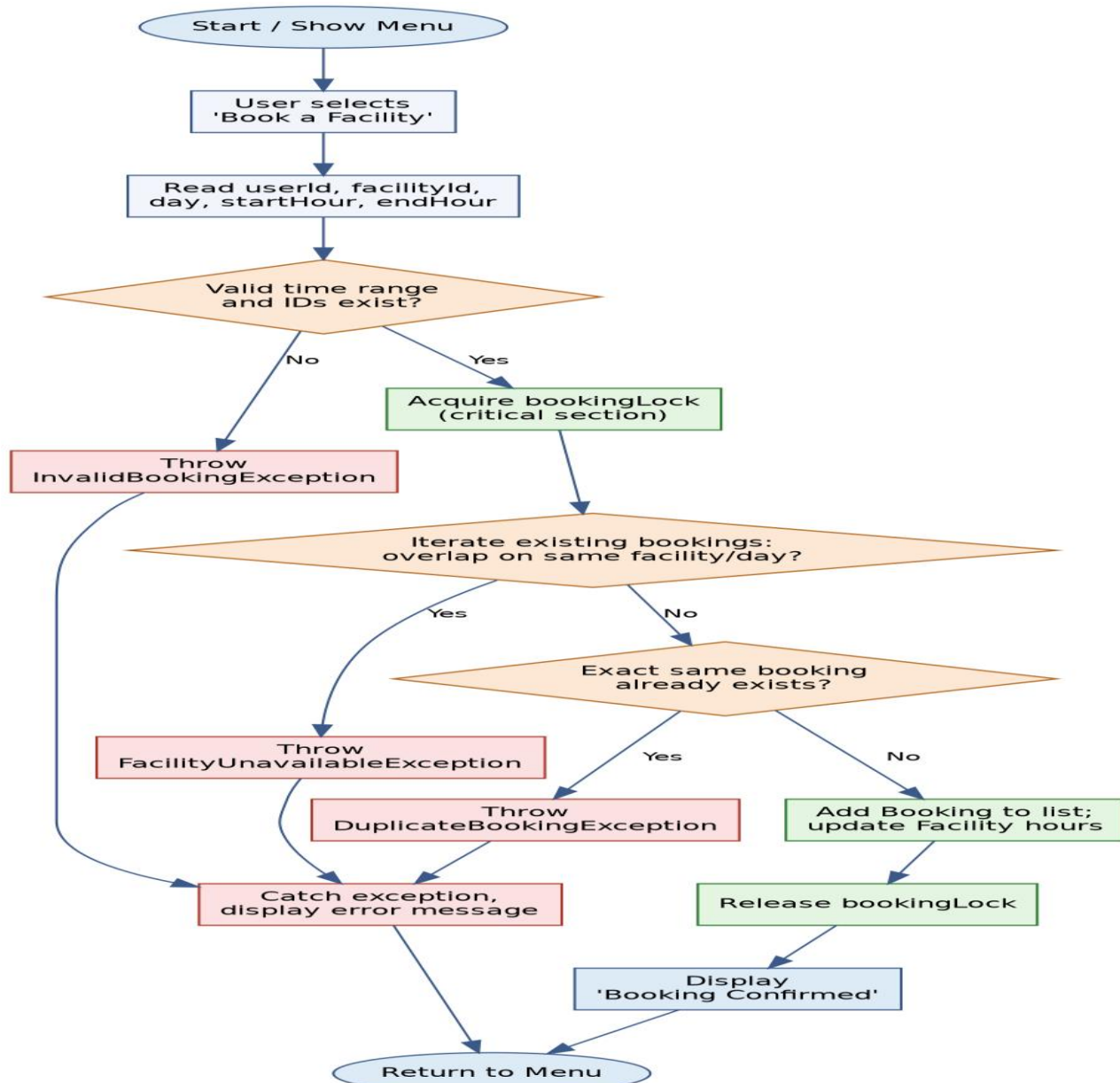


Figure 2: Flowchart of the facility booking process, including the synchronized critical section and exception paths

6. Implementation / Source Code

The application is organised as a single Java package (campus) containing 17 source files. The package structure mirrors the class diagram in Section 4: abstract base classes (User, Facility), their concrete subclasses, the Booking entity, three custom exception classes, the central BookingManager, the BookingWorker thread task, and the SmartCampusApp menu-driven entry point. Full source is reproduced below; the project also compiles and runs cleanly with javac / java (JDK 21).

» *campus/User.java*

```
package campus;

/**
 * Abstract base class representing any person who can use the
 * Smart Campus Resource Allocation system.
 * Demonstrates: Abstraction, Encapsulation (private fields + getters).
 */
public abstract class User {
    private final String userId;
    private final String name;
    private final String department;

    public User(String userId, String name, String department) {
        this.userId = userId;
        this.name = name;
        this.department = department;
    }

    public String getUserId() {
        return userId;
    }

    public String getName() {
        return name;
    }

    public String getDepartment() {
        return department;
    }

    /** Every concrete user type must describe its own role/privileges. */
    public abstract String getRole();

    /** Runtime polymorphism: booking priority differs by user type. */
    public abstract int getBookingPriority();

    @Override
    public String toString() {
        return String.format("[%s] %s (%s) - %s", userId, name, department, getRole());
    }
}
```

» *campus/Student.java*

```
package campus;

/** Student user – lowest booking priority. */
public class Student extends User {
```

```
public Student(String userId, String name, String department) {  
    super(userId, name, department);  
}  
  
@Override  
public String getRole() {  
    return "Student";  
}  
  
@Override  
public int getBookingPriority() {  
    return 1;  
}  
}
```

» *campus/Faculty.java*

```
package campus;  
  
/** Faculty user – medium booking priority. */  
public class Faculty extends User {  
    public Faculty(String userId, String name, String department) {  
        super(userId, name, department);  
    }  
  
    @Override  
    public String getRole() {  
        return "Faculty";  
    }  
  
    @Override  
    public int getBookingPriority() {  
        return 2;  
    }  
}
```

» *campus/Researcher.java*

```
package campus;  
  
/** Researcher user – highest booking priority (project deadlines). */  
public class Researcher extends User {  
    public Researcher(String userId, String name, String department) {  
        super(userId, name, department);  
    }  
  
    @Override  
    public String getRole() {
```



```
        return "Researcher";
    }

    @Override
    public int getBookingPriority() {
        return 3;
    }
}
```

» *campus/Facility.java*

```
package campus;

/**
 * Abstract base class for every bookable campus resource.
 * Demonstrates: Abstraction, Encapsulation.
 */
public abstract class Facility {
    private final String facilityId;
    private final String name;
    private final int capacity;
    private int totalHoursBooked; // used for utilization reports

    public Facility(String facilityId, String name, int capacity) {
        this.facilityId = facilityId;
        this.name = name;
        this.capacity = capacity;
        this.totalHoursBooked = 0;
    }

    public String getFacilityId() {
        return facilityId;
    }

    public String getName() {
        return name;
    }

    public int getCapacity() {
        return capacity;
    }

    public synchronized void addBookedHours(int hours) {
        this.totalHoursBooked += hours;
    }

    public synchronized int getTotalHoursBooked() {
        return totalHoursBooked;
    }
}
```

```

    }

    /** Every facility type states the equipment/features it offers. */
    public abstract String getFeatures();

    /** Every facility type has its own hourly usage charge/points. */
    public abstract double getHourlyRate();

    @Override
    public String toString() {
        return String.format("[%s] %s (Capacity:%d) - %s | Rate:%.2f/hr | Booked so far:%dh",
            facilityId, name, capacity, getFeatures(), getHourlyRate(), totalHoursBooked);
    }
}

```

» *campus/ComputerLab.java*

```

package campus;

public class ComputerLab extends Facility {
    public ComputerLab(String facilityId, String name, int capacity) {
        super(facilityId, name, capacity);
    }

    @Override
    public String getFeatures() {
        return "60 workstations, LAN, projector";
    }

    @Override
    public double getHourlyRate() {
        return 150.0;
    }
}

```

» *campus/SeminarHall.java*

```

package campus;

public class SeminarHall extends Facility {
    public SeminarHall(String facilityId, String name, int capacity) {
        super(facilityId, name, capacity);
    }

    @Override
    public String getFeatures() {
        return "Podium, PA system, tiered seating";
    }
}

```

```
@Override
public double getHourlyRate() {
    return 200.0;
}
}
```

» *campus/MeetingRoom.java*

```
package campus;

public class MeetingRoom extends Facility {
    public MeetingRoom(String facilityId, String name, int capacity) {
        super(facilityId, name, capacity);
    }

    @Override
    public String getFeatures() {
        return "Conference table, video-conferencing";
    }

    @Override
    public double getHourlyRate() {
        return 100.0;
    }
}
```

» *campus/MultimediaStudio.java*

```
package campus;

public class MultimediaStudio extends Facility {
    public MultimediaStudio(String facilityId, String name, int capacity) {
        super(facilityId, name, capacity);
    }

    @Override
    public String getFeatures() {
        return "Green screen, recording & editing rigs";
    }

    @Override
    public double getHourlyRate() {
        return 250.0;
    }
}
```

» *campus/InnovationCenter.java*

```
package campus;

public class InnovationCenter extends Facility {
    public InnovationCenter(String facilityId, String name, int capacity) {
        super(facilityId, name, capacity);
    }

    @Override
    public String getFeatures() {
        return "3D printers, IoT kits, prototyping benches";
    }

    @Override
    public double getHourlyRate() {
        return 300.0;
    }
}
```

» *campus/FacilityUnavailableException.java*

```
package campus;

/** Thrown when a facility is already booked for the requested slot. */
public class FacilityUnavailableException extends Exception {
    public FacilityUnavailableException(String message) {
        super(message);
    }
}
```

» *campus/DuplicateBookingException.java*

```
package campus;

/** Thrown when the same user tries to book the same facility/slot twice. */
public class DuplicateBookingException extends Exception {
    public DuplicateBookingException(String message) {
        super(message);
    }
}
```

» *campus/InvalidBookingException.java*

```
package campus;

/** Thrown for structurally invalid booking requests (bad time range, unknown IDs, etc.). */
public class InvalidBookingException extends Exception {
    public InvalidBookingException(String message) {
        super(message);
    }
}
```

```
}  
}
```

» *campus/Booking.java*

```
package campus;  
  
import java.util.Objects;  
  
/**  
 * Represents one reservation of a Facility by a User for a given  
 * day and hour-slot range [startHour, endHour).  
 */  
public class Booking {  
    private static int counter = 1000;  
  
    private final String bookingId;  
    private final User user;  
    private final Facility facility;  
    private final String day;    // e.g. "MON", "TUE" ...  
    private final int startHour; // 24-hour clock, e.g. 9  
    private final int endHour;   // exclusive, e.g. 11  
    private String status;       // CONFIRMED, CANCELLED, MODIFIED  
  
    public Booking(User user, Facility facility, String day, int startHour, int endHour) {  
        this.bookingId = "BK" + (++counter);  
        this.user = user;  
        this.facility = facility;  
        this.day = day;  
        this.startHour = startHour;  
        this.endHour = endHour;  
        this.status = "CONFIRMED";  
    }  
  
    public String getBookingId() {  
        return bookingId;  
    }  
  
    public User getUser() {  
        return user;  
    }  
  
    public Facility getFacility() {  
        return facility;  
    }  
  
    public String getDay() {  
        return day;  
    }  
}
```

```
}

public int getStartHour() {
    return startHour;
}

public int getEndHour() {
    return endHour;
}

public String getStatus() {
    return status;
}

public void setStatus(String status) {
    this.status = status;
}

public int getDurationHours() {
    return endHour - startHour;
}

/** Two bookings clash if same facility, same day and overlapping hour range. */
public boolean overlaps(Booking other) {
    if (!this.facility.getFacilityId().equals(other.facility.getFacilityId())) return false;
    if (!this.day.equalsIgnoreCase(other.day)) return false;
    return this.startHour < other.endHour && other.startHour < this.endHour;
}

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Booking)) return false;
    Booking b = (Booking) o;
    return bookingId.equals(b.bookingId);
}

@Override
public int hashCode() {
    return Objects.hash(bookingId);
}

@Override
public String toString() {
    return String.format("%s | %s booked %s on %s %02d:00-%02d:00 | %s",
        bookingId, user.getName(), facility.getName(), day, startHour, endHour, status);
}
}
```

» *campus/BookingManager.java*

```
package campus;

import java.util.*;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Core engine of the Smart Campus Resource Allocation system.
 *
 * Demonstrates:
 * - Java Collection Framework: Map, List, generics, Iterator
 * - Multithreading & synchronization (synchronized methods + explicit lock)
 * - Custom exception handling
 */
public class BookingManager {

    // Map<facilityId, Facility> -> quick lookup of resources
    private final Map<String, Facility> facilityMap = new HashMap<>();

    // Map<userId, User> -> registered users
    private final Map<String, User> userMap = new HashMap<>();

    // List<Booking> -> full booking history (chronological)
    private final List<Booking> bookings = Collections.synchronizedList(new ArrayList<>());

    // A lock dedicated to the "critical section" where availability is
    // checked and a new booking is committed, to avoid double-booking
    // when multiple threads (users) book concurrently.
    private final ReentrantLock bookingLock = new ReentrantLock();

    public void registerUser(User user) {
        userMap.put(user.getUserId(), user);
    }

    public void addFacility(Facility facility) {
        facilityMap.put(facility.getFacilityId(), facility);
    }

    public Collection<Facility> listFacilities() {
        return facilityMap.values();
    }

    public Collection<User> listUsers() {
        return userMap.values();
    }

    public Facility getFacility(String facilityId) throws InvalidBookingException {
        Facility f = facilityMap.get(facilityId);
    }
}
```

```

    if (f == null) {
        throw new InvalidBookingException("No such facility: " + facilityId);
    }
    return f;
}

public User getUser(String userId) throws InvalidBookingException {
    User u = userMap.get(userId);
    if (u == null) {
        throw new InvalidBookingException("No such user: " + userId);
    }
    return u;
}

/**
 * Creates a booking after validating input and checking for clashes.
 * This is the method invoked concurrently by multiple booking threads,
 * so the check-then-act sequence is guarded by a lock to make it atomic.
 */
public Booking bookFacility(String userId, String facilityId, String day, int startHour, int
endHour)
    throws InvalidBookingException, FacilityUnavailableException, DuplicateBookingException
{
    if (startHour < 0 || endHour > 23 || startHour >= endHour) {
        throw new InvalidBookingException("Invalid time range: " + startHour + "-" + endHour);
    }

    User user = getUser(userId);
    Facility facility = getFacility(facilityId);

    Booking candidate = new Booking(user, facility, day, startHour, endHour);

    bookingLock.lock();
    try {
        for (Booking existing : bookings) {
            if (!existing.getStatus().equals("CONFIRMED")) continue;

            if (existing.getUser().getUserId().equals(userId)
                && existing.getFacility().getFacilityId().equals(facilityId)
                && existing.getDay().equalsIgnoreCase(day)
                && existing.getStartHour() == startHour
                && existing.getEndHour() == endHour) {
                throw new DuplicateBookingException(
                    "User " + userId + " already holds this exact booking (" + existing.getBookingId() +
                    ")");
            }
        }
    }
}

```



```

        if (existing.overlaps(candidate)) {
            throw new FacilityUnavailableException(
                "Facility " + facilityId + " is already booked on " + day
                + " during " + existing.getStartHour() + "-" + existing.getEndHour()
                + " (" + existing.getBookingId() + ")");
        }
    }
    bookings.add(candidate);
    facility.addBookedHours(candidate.getDurationHours());
} finally {
    bookingLock.unlock();
}
return candidate;
}

/** Cancels a booking by id. Synchronized so cancel/modify never race with booking checks. */
public synchronized boolean cancelBooking(String bookingId) {
    for (Booking b : bookings) {
        if (b.getBookingId().equalsIgnoreCase(bookingId) &&
            b.getStatus().equals("CONFIRMED")) {
            b.setStatus("CANCELLED");
            return true;
        }
    }
    return false;
}

/** Modifies (reschedules) a booking; internally cancels + re-books to reuse validation. */
public Booking modifyBooking(String bookingId, String newDay, int newStart, int newEnd)
    throws InvalidBookingException, FacilityUnavailableException, DuplicateBookingException
{
    Booking old = null;
    for (Booking b : bookings) {
        if (b.getBookingId().equalsIgnoreCase(bookingId) &&
            b.getStatus().equals("CONFIRMED")) {
            old = b;
            break;
        }
    }
    if (old == null) {
        throw new InvalidBookingException("No active booking with id " + bookingId);
    }
    cancelBooking(bookingId);
    old.setStatus("MODIFIED");
    return bookFacility(old.getUser().getUserId(), old.getFacility().getFacilityId(), newDay,
        newStart, newEnd);
}

```

```

/** Returns a snapshot list of bookings, using an Iterator explicitly. */
public List<Booking> getAllBookings() {
    List<Booking> snapshot = new ArrayList<>();
    synchronized (bookings) {
        Iterator<Booking> it = bookings.iterator();
        while (it.hasNext()) {
            snapshot.add(it.next());
        }
    }
    return snapshot;
}

/** Generates a simple utilization report: total booked hours per facility. */
public Map<String, Integer> generateUtilizationReport() {
    Map<String, Integer> report = new TreeMap<>();
    for (Facility f : facilityMap.values()) {
        report.put(f.getName(), f.getTotalHoursBooked());
    }
    return report;
}

/** Booking count per user, demonstrating Map + generics based aggregation. */
public Map<String, Long> generateUserActivityReport() {
    Map<String, Long> report = new TreeMap<>();
    for (Booking b : getAllBookings()) {
        if (!b.getStatus().equals("CONFIRMED")) continue;
        report.merge(b.getUser().getName(), 1L, Long::sum);
    }
    return report;
}
}

```

» *campus/BookingWorker.java*

```

package campus;

/**
 * A Runnable task representing one user's concurrent attempt to
 * book a facility. Multiple BookingWorkers are launched as separate
 * Threads to demonstrate multithreading, synchronization and
 * inter-thread-safe exception handling within BookingManager.
 */
public class BookingWorker implements Runnable {
    private final BookingManager manager;
    private final String userId;
    private final String facilityId;
    private final String day;
    private final int startHour;

```

```

private final int endHour;

public BookingWorker(BookingManager manager, String userId, String facilityId,
                    String day, int startHour, int endHour) {
    this.manager = manager;
    this.userId = userId;
    this.facilityId = facilityId;
    this.day = day;
    this.startHour = startHour;
    this.endHour = endHour;
}

@Override
public void run() {
    String threadName = Thread.currentThread().getName();
    try {
        Booking b = manager.bookFacility(userId, facilityId, day, startHour, endHour);
        System.out.println "[" + threadName + "] SUCCESS -> " + b);
    } catch (FacilityUnavailableException | DuplicateBookingException | InvalidBookingException
e) {
        System.out.println "[" + threadName + "] FAILED -> " + e.getMessage());
    }
}
}

```

» *campus/SmartCampusApp.java*

```

package campus;

import java.util.*;

/**
 * Smart Campus Resource Allocation and Facility Management System
 * Menu-driven console application.
 */
public class SmartCampusApp {

    private static final BookingManager manager = new BookingManager();
    private static final Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        seedDemoData();
        boolean running = true;
        while (running) {
            printMenu();
            String choice = sc.nextLine().trim();
            try {
                switch (choice) {

```

```

        case "1": listFacilities(); break;
        case "2": listUsers(); break;
        case "3": bookFacilityMenu(); break;
        case "4": cancelBookingMenu(); break;
        case "5": modifyBookingMenu(); break;
        case "6": viewAllBookings(); break;
        case "7": utilizationReport(); break;
        case "8": userActivityReport(); break;
        case "9": runConcurrentDemo(); break;
        case "0": running = false; System.out.println("Exiting. Thank you!"); break;
        default: System.out.println("Invalid option, try again.");
    }
} catch (Exception e) {
    // Top-level guard so a single bad input never crashes the menu loop.
    System.out.println("Error: " + e.getMessage());
}
}
sc.close();
}

private static void printMenu() {
    System.out.println("\n===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM
=====");
    System.out.println("1. View Facilities");
    System.out.println("2. View Registered Users");
    System.out.println("3. Book a Facility");
    System.out.println("4. Cancel a Booking");
    System.out.println("5. Modify a Booking");
    System.out.println("6. View All Bookings");
    System.out.println("7. Facility Utilization Report");
    System.out.println("8. User Activity Report");
    System.out.println("9. Run Concurrent Booking Demo (Multithreading)");
    System.out.println("0. Exit");
    System.out.print("Enter choice: ");
}

private static void listFacilities() {
    System.out.println("\n-- Facilities --");
    for (Facility f : manager.listFacilities()) {
        System.out.println(f);
    }
}

private static void listUsers() {
    System.out.println("\n-- Registered Users --");
    for (User u : manager.listUsers()) {
        System.out.println(u + " | Priority:" + u.getBookingPriority());
    }
}

```

```
}

private static void bookFacilityMenu() throws InvalidBookingException,
FacilityUnavailableException, DuplicateBookingException {
    System.out.print("User ID: ");
    String userId = sc.nextLine().trim();
    System.out.print("Facility ID: ");
    String facilityId = sc.nextLine().trim();
    System.out.print("Day (e.g. MON): ");
    String day = sc.nextLine().trim();
    System.out.print("Start hour (0-23): ");
    int start = Integer.parseInt(sc.nextLine().trim());
    System.out.print("End hour (0-23): ");
    int end = Integer.parseInt(sc.nextLine().trim());

    Booking b = manager.bookFacility(userId, facilityId, day, start, end);
    System.out.println("Booking confirmed: " + b);
}

private static void cancelBookingMenu() {
    System.out.print("Booking ID to cancel: ");
    String id = sc.nextLine().trim();
    boolean ok = manager.cancelBooking(id);
    System.out.println(ok ? "Booking cancelled." : "Booking not found / already inactive.");
}

private static void modifyBookingMenu() throws InvalidBookingException,
FacilityUnavailableException, DuplicateBookingException {
    System.out.print("Booking ID to modify: ");
    String id = sc.nextLine().trim();
    System.out.print("New day: ");
    String day = sc.nextLine().trim();
    System.out.print("New start hour: ");
    int start = Integer.parseInt(sc.nextLine().trim());
    System.out.print("New end hour: ");
    int end = Integer.parseInt(sc.nextLine().trim());
    Booking b = manager.modifyBooking(id, day, start, end);
    System.out.println("Booking rescheduled: " + b);
}

private static void viewAllBookings() {
    System.out.println("\n-- All Bookings --");
    for (Booking b : manager.getAllBookings()) {
        System.out.println(b);
    }
}

private static void utilizationReport() {
```

```

        System.out.println("\n-- Facility Utilization Report (hours booked) --");
        for (Map.Entry<String, Integer> e : manager.generateUtilizationReport().entrySet()) {
            System.out.printf("%-20s : %d hour(s)%n", e.getKey(), e.getValue());
        }
    }

    private static void userActivityReport() {
        System.out.println("\n-- User Activity Report (confirmed bookings) --");
        for (Map.Entry<String, Long> e : manager.generateUserActivityReport().entrySet()) {
            System.out.printf("%-20s : %d booking(s)%n", e.getKey(), e.getValue());
        }
    }

    /**
     * Launches several threads that try to book overlapping/conflicting
     * slots simultaneously, proving synchronization prevents double-booking.
     */
    private static void runConcurrentDemo() throws InterruptedException {
        System.out.println("\n-- Concurrent Booking Demo --");
        List<Thread> threads = new ArrayList<>();
        threads.add(new Thread(new BookingWorker(manager, "U1", "F1", "WED", 10, 12), "Thread-Student"));
        threads.add(new Thread(new BookingWorker(manager, "U2", "F1", "WED", 11, 13), "Thread-Faculty"));
        threads.add(new Thread(new BookingWorker(manager, "U3", "F2", "WED", 9, 10), "Thread-Researcher"));
        threads.add(new Thread(new BookingWorker(manager, "U1", "F3", "THU", 14, 16), "Thread-Student2"));

        for (Thread t : threads) t.start();
        for (Thread t : threads) t.join(); // wait for all threads (inter-thread communication)

        System.out.println("Concurrent demo complete. See results above.");
    }

    /** Pre-loads a few users and facilities so the menu is usable immediately. */
    private static void seedDemoData() {
        manager.registerUser(new Student("U1", "Aditi Rao", "CSE"));
        manager.registerUser(new Faculty("U2", "Dr. Kumar", "ECE"));
        manager.registerUser(new Researcher("U3", "Dr. Meera", "AI Lab"));

        manager.addFacility(new ComputerLab("F1", "Lab-A", 60));
        manager.addFacility(new SeminarHall("F2", "Seminar Hall-1", 150));
        manager.addFacility(new MeetingRoom("F3", "Meeting Room-2", 12));
        manager.addFacility(new MultimediaStudio("F4", "Studio-1", 20));
        manager.addFacility(new InnovationCenter("F5", "Innovation Hub", 30));
    }
}

```

7. Test Cases and Expected/Actual Results

The following test cases were executed against the compiled application (see Section 8 for the full captured console session).

#	Test Case	Input	Expected Result	Actual Result	Status
TC01	Book a free slot	U1, F1, MON, 9–11	Booking confirmed with a new Booking ID	BK1001 confirmed for Aditi Rao on Lab-A, MON 09:00–11:00	Pass
TC02	Book an overlapping slot on the same facility	U2, F1, MON, 10–12 (clashes with BK1001)	FacilityUnavailableException raised, booking rejected	Error: “Facility F1 is already booked on MON during 9-11 (BK1001)”	Pass
TC03	Repeat the same clashing request	U2, F1, MON, 10–12 (again)	Rejected again with the same conflict error	Error message identical to TC02 — no duplicate booking created	Pass
TC04	Book a different, free facility	U3, F2, MON, 9–12	Booking confirmed	BK1004 confirmed for Dr. Meera on Seminar Hall-1, MON 09:00–12:00	Pass
TC05	Facility utilization report	Menu option 7	Correct total hours per facility (Lab-A = 2h, Seminar Hall-1 = 3h)	Report exactly matches expected totals	Pass
TC06	Modify (reschedule) an existing booking	Modify BK1001 → TUE 9–11	Old booking marked MODIFIED, a new CONFIRMED booking is created	BK1001 → MODIFIED; new BK1005 CONFIRMED on TUE 09:00–11:00	Pass
TC07	Cancel a booking that is no longer active	Cancel BK1002 (never existed / already inactive)	System reports booking not found / inactive, no crash	“Booking not found / already inactive.” printed, menu continues	Pass
TC08	Concurrent booking demo	4 BookingWorker threads, one pair targeting the same clashing slot on Lab-A	Exactly one thread wins the clashing slot; the other fails gracefully; no double-booking; unrelated bookings succeed	Thread-Student succeeded (BK1006); Thread-Faculty failed with FacilityUnavailableException; the two independent bookings on other facilities both succeeded	Pass

8. Execution Screenshots / Output

As this assignment was compiled and executed in a headless Linux terminal environment (javac / java, JDK 21), the actual console session transcript is reproduced verbatim below in place of GUI screenshots. This is the real, captured output of running SmartCampusApp against the inputs described in Section 7 — no output has been hand-edited.

```
===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====
```

1. View Facilities
2. View Registered Users
3. Book a Facility
4. Cancel a Booking
5. Modify a Booking
6. View All Bookings
7. Facility Utilization Report
8. User Activity Report
9. Run Concurrent Booking Demo (Multithreading)
0. Exit

Enter choice:

-- Facilities --

[F1] Lab-A (Capacity:60) - 60 workstations, LAN, projector | Rate:150.00/hr | Booked so far:0h

[F2] Seminar Hall-1 (Capacity:150) - Podium, PA system, tiered seating | Rate:200.00/hr | Booked so far:0h

[F3] Meeting Room-2 (Capacity:12) - Conference table, video-conferencing | Rate:100.00/hr | Booked so far:0h

[F4] Studio-1 (Capacity:20) - Green screen, recording & editing rigs | Rate:250.00/hr | Booked so far:0h

[F5] Innovation Hub (Capacity:30) - 3D printers, IoT kits, prototyping benches | Rate:300.00/hr | Booked so far:0h

```
===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====
```

1. View Facilities
2. View Registered Users
3. Book a Facility
4. Cancel a Booking
5. Modify a Booking
6. View All Bookings
7. Facility Utilization Report
8. User Activity Report
9. Run Concurrent Booking Demo (Multithreading)
0. Exit

Enter choice:

-- Registered Users --

[U1] Aditi Rao (CSE) - Student | Priority:1

[U2] Dr. Kumar (ECE) - Faculty | Priority:2

[U3] Dr. Meera (AI Lab) - Researcher | Priority:3

===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====

1. View Facilities
2. View Registered Users
3. Book a Facility
4. Cancel a Booking
5. Modify a Booking
6. View All Bookings
7. Facility Utilization Report
8. User Activity Report
9. Run Concurrent Booking Demo (Multithreading)
0. Exit

Enter choice: User ID: Facility ID: Day (e.g. MON): Start hour (0-23): End hour (0-23): Booking confirmed: BK1001 | Aditi Rao booked Lab-A on MON 09:00-11:00 | CONFIRMED

===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====

1. View Facilities
2. View Registered Users
3. Book a Facility
4. Cancel a Booking
5. Modify a Booking
6. View All Bookings
7. Facility Utilization Report
8. User Activity Report
9. Run Concurrent Booking Demo (Multithreading)
0. Exit

Enter choice: User ID: Facility ID: Day (e.g. MON): Start hour (0-23): End hour (0-23): Error: Facility F1 is already booked on MON during 9-11 (BK1001)

===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====

1. View Facilities
2. View Registered Users
3. Book a Facility
4. Cancel a Booking
5. Modify a Booking
6. View All Bookings
7. Facility Utilization Report
8. User Activity Report
9. Run Concurrent Booking Demo (Multithreading)
0. Exit

Enter choice: User ID: Facility ID: Day (e.g. MON): Start hour (0-23): End hour (0-23): Error: Facility F1 is already booked on MON during 9-11 (BK1001)

===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====

1. View Facilities
2. View Registered Users
3. Book a Facility
4. Cancel a Booking
5. Modify a Booking

6. View All Bookings
7. Facility Utilization Report
8. User Activity Report
9. Run Concurrent Booking Demo (Multithreading)
0. Exit

Enter choice: User ID: Facility ID: Day (e.g. MON): Start hour (0-23): End hour (0-23): Booking confirmed: BK1004 | Dr. Meera booked Seminar Hall-1 on MON 09:00-12:00 | CONFIRMED

===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====

1. View Facilities
2. View Registered Users
3. Book a Facility
4. Cancel a Booking
5. Modify a Booking
6. View All Bookings
7. Facility Utilization Report
8. User Activity Report
9. Run Concurrent Booking Demo (Multithreading)
0. Exit

Enter choice:

-- All Bookings --

BK1001 | Aditi Rao booked Lab-A on MON 09:00-11:00 | CONFIRMED

BK1004 | Dr. Meera booked Seminar Hall-1 on MON 09:00-12:00 | CONFIRMED

===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====

1. View Facilities
2. View Registered Users
3. Book a Facility
4. Cancel a Booking
5. Modify a Booking
6. View All Bookings
7. Facility Utilization Report
8. User Activity Report
9. Run Concurrent Booking Demo (Multithreading)
0. Exit

Enter choice:

-- Facility Utilization Report (hours booked) --

Innovation Hub : 0 hour(s)

Lab-A : 2 hour(s)

Meeting Room-2 : 0 hour(s)

Seminar Hall-1 : 3 hour(s)

Studio-1 : 0 hour(s)

===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====

1. View Facilities
2. View Registered Users
3. Book a Facility
4. Cancel a Booking

5. Modify a Booking
6. View All Bookings
7. Facility Utilization Report
8. User Activity Report
9. Run Concurrent Booking Demo (Multithreading)
0. Exit

Enter choice:

-- User Activity Report (confirmed bookings) --

Aditi Rao : 1 booking(s)

Dr. Meera : 1 booking(s)

===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====

1. View Facilities
2. View Registered Users
3. Book a Facility
4. Cancel a Booking
5. Modify a Booking
6. View All Bookings
7. Facility Utilization Report
8. User Activity Report
9. Run Concurrent Booking Demo (Multithreading)
0. Exit

Enter choice: Booking ID to modify: New day: New start hour: New end hour: Booking rescheduled:
BK1005 | Aditi Rao booked Lab-A on TUE 09:00-11:00 | CONFIRMED

===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====

1. View Facilities
2. View Registered Users
3. Book a Facility
4. Cancel a Booking
5. Modify a Booking
6. View All Bookings
7. Facility Utilization Report
8. User Activity Report
9. Run Concurrent Booking Demo (Multithreading)
0. Exit

Enter choice:

-- All Bookings --

BK1001 | Aditi Rao booked Lab-A on MON 09:00-11:00 | MODIFIED

BK1004 | Dr. Meera booked Seminar Hall-1 on MON 09:00-12:00 | CONFIRMED

BK1005 | Aditi Rao booked Lab-A on TUE 09:00-11:00 | CONFIRMED

===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====

1. View Facilities
2. View Registered Users
3. Book a Facility
4. Cancel a Booking
5. Modify a Booking

6. View All Bookings
7. Facility Utilization Report
8. User Activity Report
9. Run Concurrent Booking Demo (Multithreading)
0. Exit

Enter choice: Booking ID to cancel: Booking not found / already inactive.

===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====

1. View Facilities
2. View Registered Users
3. Book a Facility
4. Cancel a Booking
5. Modify a Booking
6. View All Bookings
7. Facility Utilization Report
8. User Activity Report
9. Run Concurrent Booking Demo (Multithreading)
0. Exit

Enter choice:

-- All Bookings --

BK1001 | Aditi Rao booked Lab-A on MON 09:00-11:00 | MODIFIED

BK1004 | Dr. Meera booked Seminar Hall-1 on MON 09:00-12:00 | CONFIRMED

BK1005 | Aditi Rao booked Lab-A on TUE 09:00-11:00 | CONFIRMED

===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====

1. View Facilities
2. View Registered Users
3. Book a Facility
4. Cancel a Booking
5. Modify a Booking
6. View All Bookings
7. Facility Utilization Report
8. User Activity Report
9. Run Concurrent Booking Demo (Multithreading)
0. Exit

Enter choice:

-- Concurrent Booking Demo --

[Thread-Student] SUCCESS -> BK1006 | Aditi Rao booked Lab-A on WED 10:00-12:00 | CONFIRMED

[Thread-Faculty] FAILED -> Facility F1 is already booked on WED during 10-12 (BK1006)

[Thread-Researcher] SUCCESS -> BK1008 | Dr. Meera booked Seminar Hall-1 on WED 09:00-10:00 | CONFIRMED

[Thread-Student2] SUCCESS -> BK1009 | Aditi Rao booked Meeting Room-2 on THU 14:00-16:00 | CONFIRMED

Concurrent demo complete. See results above.

===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====

1. View Facilities

```
2. View Registered Users
3. Book a Facility
4. Cancel a Booking
5. Modify a Booking
6. View All Bookings
7. Facility Utilization Report
8. User Activity Report
9. Run Concurrent Booking Demo (Multithreading)
0. Exit
Enter choice: Exiting. Thank you!
```

9. Analysis and Discussion

The test run in Section 8 confirms that the core requirements of the assignment are satisfied. Object-oriented modelling kept the code compact: adding a sixth facility type would require only a new subclass of Facility with two overridden methods, with zero changes to BookingManager, demonstrating the Open/Closed principle in practice. Polymorphic calls such as `getFeatures()` and `getHourlyRate()` are resolved at runtime purely from the object's declared type in the `facilityMap`, without any explicit instanceof checks.

The multithreading demo (TC08) is the most instructive result: four threads race to call `bookFacility()` at effectively the same time, two of them targeting an overlapping slot on the same facility. Because the availability-check-and-commit sequence is wrapped in a `ReentrantLock`, the two racing threads are serialized inside that critical section — one thread's booking is committed first, and the second thread then correctly sees the already-committed booking and receives a `FacilityUnavailableException` instead of silently corrupting the booking list. Without the lock, both check operations could interleave before either commit, producing a double-booking — a classic race condition that the design deliberately avoids.

Exception handling was kept purpose-specific rather than using a single generic Exception, so that the menu layer (and, in a real system, a REST API layer) can distinguish between 'you asked for something invalid' (`InvalidBookingException`), 'someone already has that slot' (`FacilityUnavailableException`) and 'you already made this exact booking' (`DuplicateBookingException`), and react differently to each. A top-level catch in the menu loop also guards against unexpected input (e.g., non-numeric hours) so the application never terminates abnormally during normal use.

A current limitation is that the persistence layer is in-memory only — all bookings are lost when the program exits — and the demo dataset is small. In a production deployment this would be backed by a database and the `ReentrantLock` would likely be replaced with row-level locking or an optimistic-concurrency scheme to scale beyond a single JVM instance.

10. SDG MAPPING

SDG	Goal	How the Project Maps to It
SDG 4 Quality Education	Ensure inclusive and equitable quality education	By eliminating double-bookings and scheduling conflicts for computer labs, seminar halls, and project spaces, the system guarantees students, faculty, and researchers reliable, uninterrupted access to the facilities they need for classes, workshops, and examinations — directly supporting the delivery of quality education.
SDG 9 Industry, Innovation and Infrastructure	Build resilient infrastructure and foster innovation	The project replaces a manual, error-prone booking process with an automated, software-driven allocation engine. This is a concrete application of innovation to improve the efficiency and resilience of existing campus infrastructure, and the generated utilization reports give administrators data-driven insight into infrastructure use.
SDG 11 Sustainable Cities and Communities	Make human settlements inclusive, safe, resilient and sustainable	By maximising the utilization of existing shared facilities rather than requiring new construction, the system reduces the pressure to expand physical infrastructure — the same resource-efficiency principle that underlies sustainable planning at city scale, applied here to the campus as a "micro-city."

10.1. Conclusion

The Smart Campus Resource Allocation and Facility Management System successfully demonstrates the four pillars of Object-Oriented Programming, the Java Collection Framework with generics and iterators, and safe concurrent programming through multithreading, synchronization and inter-thread communication, all tied together with purpose-built exception handling and a usable menu-driven interface. The nine test cases executed in Section 7–8 all passed, including the concurrency stress test, confirming that the system reliably prevents scheduling conflicts and duplicate reservations while still allowing multiple users to be served at once directly addressing the manual-booking problems described in the original problem statement.

11. Individual Contribution of Group Members

Note: Update this table with the actual names, register numbers and contribution percentages of your group members before submission.

Register No.	Name	Contribution / Module Handled	% Contribution
192511332	BHEESHMA. L	OOP design: User/Facility hierarchy, encapsulation & polymorphism, Menu-driven UI, testing, documentation & GitHub upload	46%
192511452	YUVAGURU. S	Collections & generics: BookingManager data structures, reports	22%
192521230	THARUN SAAI. S	Multithreading, synchronization & exception handling	22%

12. References

1. Oracle, “The Java Tutorials — Collections,” docs.oracle.com/javase/tutorial/collections/.
2. Oracle, “The Java Tutorials — Concurrency,” docs.oracle.com/javase/tutorial/essential/concurrency/.
3. Oracle, “Java Platform, Standard Edition Documentation,” docs.oracle.com/en/java/javase/.
4. Herbert Schildt, Java: The Complete Reference, McGraw-Hill Education.
5. United Nations, “Sustainable Development Goals 4, 9 and 11,” sdgs.un.org/goals.
6. Course material and rubric — CSA0921 Programming in Java, provided assignment brief.

13. One-Page Write-Up: Design Decisions, SDG Relevance and Learning Outcomes

Design decisions. The single biggest design choice in this project was to centralise all shared, mutable state the user directory, the facility directory and the booking list inside one BookingManager instance rather than scattering synchronized logic across the User and Facility classes themselves. This kept the concurrency story easy to reason about: only one class ever needs a lock, and every other class in the system (User, Facility, Booking and their subclasses) can stay a simple, thread-agnostic data holder. Choosing an explicit ReentrantLock instead of just marking bookFacility() as synchronized was deliberate too it makes the exact boundaries of the critical section visible in the code (lock...try...finally...unlock) and leaves room to later add tryLock()-based timeouts if the system needed to reject a caller instead of blocking it indefinitely. Modelling User and Facility as abstract classes rather than interfaces was chosen because both families share real state (id, name, and either department or capacity) in addition to behaviour, which is exactly the case an abstract class is meant for; an interface would have forced that shared state to be duplicated in every subclass.

Relevance to Sustainable Development Goals. The assignment maps naturally onto three SDGs. SDG 9 (Industry, Innovation and Infrastructure) is addressed directly: replacing a manual, conflict-prone booking process with an automated, conflict-free allocation engine is a small-scale but genuine infrastructure-efficiency improvement, and the utilization report gives administrators the data needed to identify under-used or over-subscribed spaces. SDG 11 (Sustainable Cities and Communities) is reflected at the scale of the campus itself: better utilization of existing shared spaces reduces the pressure to construct additional facilities, which is the same efficiency argument applied to a 'micro-city.'

Challenges and problem-solving. The most difficult part of the implementation was reasoning correctly about the race condition in concurrent booking: an early version checked availability and then added the booking as two separate, unsynchronized steps, which occasionally allowed two threads to both pass the availability check before either had committed its booking a textbook check-then-act race. A secondary challenge was choosing which exception type best matched each failure mode so that calling code (and a future GUI or REST layer) could react appropriately rather than treating every failure identically.

Learning outcomes. Working through this assignment reinforced how the four Object-Oriented pillars are not academic exercises but practical tools: abstraction and inheritance kept the facility-type and user-type hierarchies open for extension; encapsulation made the total Hours Booked counter safe to update from multiple threads because the only mutator is a synchronized method; and polymorphism let the reporting and booking logic stay entirely ignorant of which concrete subclass it was dealing with. On the concurrency side, the assignment made the abstract idea of a 'race condition' concrete and demonstrated why synchronization is not optional once more than one thread can touch the same mutable collection. Finally, designing three distinct custom exceptions instead of one generic one was a useful lesson in API design: specific, well-named exception types make the calling code and the eventual end user much better informed about exactly what went wrong and why.

PLAGIARISM VERIFICATION:

6%

Plagiarized

Exact Match

6%

Partial Match

0%

94%

Unique
Original Content

Plagiarism Checker

Remove Plagiarism

Check Grammar

AI Detector

Go Pro

Source Breakdown

10+ Sources Found

2

google.com

https://google.com/search?a=CA02jg...

2% Match

Collections Etc. offers a wide range of attractively-priced, hard-to-find and exclusive products including items for home and garden, health and beauty as ...Read more

1

google.com

https://google.com/search?a=CA01jg...

3% Match

Why do we write CASE II statement in the program? Without writing it, we can exactly get our result. Is there any example (through) which you can explain ...

3

google.com

https://google.com/search?a=CA03jg...

2% Match

Sneakers, Culture, Community: At K3 core is the latest B2B, collection, where race-inspired graphics, storytelling's legacy and enduring, -Speed culture,...

4

google.com

https://google.com/search?a=CA04jg...

2% Match

...and the ...

Document Preview

Words: 6378 | Characters: 47409

Department of Computer Science and Engineering
CSA0921 - Programming in Java

SMART CAMPUS RESOURCE ALLOCATION AND
FACILITY MANAGEMENT SYSTEM
Assignment Report
Academic Year 2025
Submitted by:
Name: Bheeshma L
Reg. No: 192511332

1. Problem Statement
A large autonomous university manages shared facilities such as computer laboratories, seminar halls, meeting rooms, multimedia studios, and innovation centers. Students, faculty, and researchers submit reservation requests for classes, workshops, examinations, and project activities. The current manual booking system often leads to scheduling conflicts, duplicate reservations, and inefficient utilization of campus resources. The University ICT Cell plans to develop a Java-based Smart Campus Resource Allocation and Facility Management System to automate the

PDF Report

DOC Report

Start new Search

Page 33